Work Order

Suggested order of work, made from `docs/SRS.md` and `docs/CLASSES_SPECS.md` with the `manage-project` skill. It is a suggestion only: no people, dates or sprints, and no code.

How to read it: Phase → Feature (FRs + BRs + NFRs + needs) → Cycle (one action on one entity) → Tasks (DAL → BLL → PL → Check).

Two notes on fit with `AGENTS.md`:

- "Simplest way" here means the simplest way **inside** the `AGENTS.md` architecture (Repository, Unit of Work, services, AutoMapper, FluentValidation). No layers or packages beyond those.
- The team decided on **no automated tests**. So the last task of each cycle is a **manual check** of its BRs in the running app, not a test project.

Overview

Phase	Features	FRs
1 Foundations	Project foundation, Log in and roles, Tourist sign-up, Guide sign-up	FR-01, FR-02, FR-03
2 Core flow	Verification, guide status, profile, search, public profile, booking request, guide answers, my bookings, payment	FR-04 – FR-16
3 Secondary	Scheduled job, reviews	FR-17 – FR-20
4 Later	Out-of-scope items (SRS section 6)	—

Dependency chain

```
Foundation → FR-03 → FR-01 → FR-02 → FR-04/05/06 → FR-07
                                                  └→ FR-08 → FR-10, FR-09 → FR-11/12 → FR-13/16 → FR-14 → FR-17/18 → FR-19/20
                                                                                                  └→ FR-15
```

Cross-cutting NFRs (placed where first needed)

NFR	What	First needed in
NFR-07 English UI · NFR-08 ASP.NET Core MVC + Git	Project shape	1.0 Foundation
Validation (4 layers, `AGENTS.md`) · error handling · anti-forgery	All forms	1.0 Foundation
NFR-01 Role access	Every area	1.1 Log in and roles
NFR-03 Hashed passwords	Accounts	1.2 Tourist sign-up
NFR-02 Documents outside `wwwroot`	File upload	1.3 Guide sign-up
NFR-05 Allowed status changes only	Booking state machine	2.6 Booking request
NFR-04 No card data stored	Payment	2.9 Payment
NFR-06 Scheduled job	Auto status changes	3.1 Scheduled job

Phase 1 — Foundations

1.0 Project foundation

- **FRs:** none (everything else needs it) · **NFRs:** NFR-07, NFR-08, validation, error handling · **needs:** —

Cycle 1.0.a — Data model

1. DAL: 10 entities in `Entities/` (Class Specs 1–7); enums already done
2. DAL: one `IEntityTypeConfiguration<T>` per entity (keys, lengths, indexes, check constraints, `Restrict` deletes, `RowVersion`)
3. DAL: `AppDbContext : IdentityDbContext<ApplicationUser>` with all `DbSet`s
4. DAL: `IRepository<T>` / `Repository<T>`, empty specific repository interfaces and classes, `IUnitOfWork` / `UnitOfWork`, `PagedList<T>`, `AddDataAccess()`
5. DAL: migration `InitialCreate`
6. Check: `dotnet build` passes; `dotnet ef database update` creates every table with the right keys and constraints

Cycle 1.0.b — App skeleton

1. Create `TourGuide.sln`, `TourGuide.BLL`, `TourGuide.PL`; add project references (Web → BLL → DAL)
2. BLL: `Result`, `Result<T>`, `ValidationError`, `ErrorCodes`, `BookingRules`, `FileStorageOptions`, `BookingJobOptions`, `AddBusinessLogic()` (validators + AutoMapper)
3. PL: `Program.cs` (SQL Server, Identity, `AddDataAccess`, `AddBusinessLogic`, `AddAutoMapper`, global anti-forgery, error handler, `AssertConfigurationIsValid` in Development)
4. PL: shared layout from the UI design, `_ValidationScriptsPartial`, `ICurrentUserService`, `ModelStateExtensions`, error page
5. Check: app starts, home page shows the layout, a thrown error shows the friendly error page

Cycle 1.0.c — Seed data

1. DAL: seed the 3 roles
2. DAL: seed admin account(s) — **blocked:** emails not decided (`STATE.md` → Known issues)
3. DAL: seed languages and specializations — **blocked:** lists not decided
4. Check: roles and seed rows exist after `database update`

1.1 Log in, log out and role access — FR-03

- **BRs:** — · **NFRs:** NFR-01 · **needs:** 1.0

Cycle 1.1.a — Log in / log out

1. PL: `LoginViewModel` + DataAnnotations; `AccountController` Login GET/POST, Logout POST (`SignInManager`)
2. PL: Login view (UI screen 2 tab or separate page)
3. Check: seeded admin can log in and out; wrong password shows an error

Cycle 1.1.b — Role access

1. PL: Areas `Admin`, `Guide`, `Tourist` with `[Authorize(Roles = ...)]` base controllers; access-denied page
2. PL: menu shows only the links of the user's role
3. Check (NFR-01): a Tourist opening an Admin URL gets access denied

1.2 Tourist sign-up — FR-01

- **BRs:** — · **NFRs:** NFR-03 · **needs:** 1.1

Cycle 1.2.a — Add Tourist

1. BLL: `ICountryProvider` + `CountryProvider`
2. BLL: `RegisterTouristRequest`, `RegisterTouristValidator`, `IAccountService.RegisterTouristAsync` (UserManager, Tourist role)
3. PL: `RegisterTouristViewModel` + attributes; `AccountController` Register GET/POST; view (UI screen 2)
4. Check: new tourist can sign up and log in; duplicate email and bad country are refused; password is stored hashed

1.3 Guide sign-up with documents — FR-02

- **BRs:** BR-01 (starts Pending), BR-03 · **NFRs:** NFR-02 · **needs:** 1.2

Cycle 1.3.a — Add Guide

1. BLL: `IFileStorage` + `LocalFileStorage` (folder outside `wwwroot`), `FileUpload`, `FileUploadValidator` (content signature, ≤ 5 MB)
2. BLL: `RegisterGuideRequest`, `RegisterGuideValidator`, `IAccountService.RegisterGuideAsync` (user + Guide role + `GuideProfile` Pending + 3 `GuideDocument` in one transaction)
3. PL: `MaxFileSizeAttribute`, `AllowedFileTypesAttribute` + jQuery adapters in `wwwroot/js/validation.js`
4. PL: `RegisterGuideViewModel`; `AccountController` RegisterGuide GET/POST; view with 3 uploads
5. Check: guide signs up with 3 valid files → status Pending (BR-01); 6 MB file or `.exe` renamed to `.pdf` is refused; files are not reachable by URL (NFR-02)

Phase 2 — Core flow

2.1 Guide verification — FR-04, FR-05, FR-06

- **BRs:** BR-01, BR-03 · **NFRs:** NFR-01, NFR-02 · **needs:** FR-02

Cycle 2.1.a — List pending guides (FR-04)

1. DAL: `IGuideProfileRepository.GetPendingAsync`
2. BLL: `PendingGuideDto`, `GuideMappingProfile`, `IGuideVerificationService.GetPendingAsync`
3. PL: Admin `GuideVerificationController.Index`, `PendingGuidesViewModel`, view
4. Check: only Pending guides show; a non-admin is denied

Cycle 2.1.b — Guide details and documents (FR-05)

1. DAL: `GetWithDetailsAsync`, `IGuideDocumentRepository.GetByGuideAsync`
2. BLL: `GuideReviewDto`, `GetForReviewAsync`, `OpenDocumentAsync`
3. PL: `Review(id)` view; `Document(documentId)` streams the file
4. Check (BR-03): admin can open each document; a guide or tourist opening the document URL is denied

Cycle 2.1.c — Approve (FR-06)

1. BLL: `ApproveAsync` (only from Pending, all 3 documents present; sets reviewer and time)
2. PL: Approve POST button on the Review view
3. Check: approved guide leaves the pending list; approve with a missing document is refused

Cycle 2.1.d — Reject (FR-06)

1. BLL: `RejectGuideRequest`, `RejectGuideValidator` (10–500), `RejectAsync`
2. PL: `RejectGuideViewModel`, reject form with reason
3. Check: reason under 10 characters is refused on client and server; rejected guide leaves the list

2.2 Guide status and re-apply — FR-07

- **BRs:** BR-01 · **NFRs:** NFR-01, NFR-02 · **needs:** FR-06

Cycle 2.2.a — Status details

1. BLL: `GuideStatusDto`, `GetMyStatusAsync`
2. PL: Guide area `VerificationController.Status`, view (Pending / Rejected with reason / Approved)
3. Check (BR-01): a Pending or Rejected guide sees only this page in the Guide area

Cycle 2.2.b — Re-apply (edit documents)

1. DAL: `IGuideDocumentRepository.GetByTypeAsync`
2. BLL: `ReapplyRequest`, `ReapplyValidator`, `ReapplyAsync` (only from Rejected; replace files; back to Pending; clear review fields)
3. PL: `ReapplyViewModel`, form with 3 uploads
4. Check: re-apply works only from Rejected; old files are replaced; guide is back in the admin pending list

2.3 Guide profile — FR-08

- **BRs:** BR-02 (profile complete) · **NFRs:** NFR-01 · **needs:** FR-06

Cycle 2.3.a — Edit profile

1. DAL: `GetByUserIdAsync` (with languages and specializations); `Languages`, `Specializations` on the Unit of Work
2. BLL: `GuideProfileEditDto`, `UpdateGuideProfileRequest`, `UpdateGuideProfileValidator`, `GetMyProfileAsync`, `UpdateMyProfileAsync` (Approved only; replace language and specialization lists; save photo)
3. PL: Guide `ProfileController` Edit GET/POST, `EditGuideProfileViewModel`, view; `Photo(id)` action
4. Check: price 0 or 1,001 is refused; no language is refused; a Pending guide cannot edit; a profile with all required fields counts as complete

2.4 Search guides — FR-10

- **BRs:** BR-02 · **NFRs:** — · **needs:** FR-08

Cycle 2.4.a — List guides with filters

1. DAL: `GuideSearchQuery`, `IGuideProfileRepository.SearchAsync` (always Approved + complete; filters; paging)
2. BLL: `GuideSearchFilter`, `GuideSearchFilterValidator`, `GuideCardDto`, `SearchFilterOptionsDto`, `ISearchService`, `LookupMappingProfile`
3. PL: `SearchController.Index`, `SearchViewModel`, view (UI screen 5 + filter sidebar from screen 8)
4. Check (BR-02): Pending, Rejected and incomplete guides never show; each filter and the price range work; min > max is refused

2.5 Public guide profile — FR-09

- **BRs:** — · **NFRs:** — · **needs:** FR-08

Cycle 2.5.a — Guide details

1. DAL: `IReviewRepository.GetForGuideAsync` (paged)
2. BLL: `PublicGuideProfileDto`, `ReviewDto`, `GetPublicProfileAsync`
3. PL: `GuidesController.Details`, `PublicGuideProfileViewModel`, view with rating, reviews and a Book button
4. Check: page shows details and rating 0 with no reviews; a non-bookable guide's page is not found

2.6 Booking request — FR-11, FR-12

- **BRs:** BR-02, BR-04, BR-11 · **NFRs:** NFR-05 · **needs:** FR-09

Cycle 2.6.a — Price preview (FR-12)

1. BLL: `IPriceCalculator` + `PriceCalculator` (billable hours, total; 8 h per day)
2. BLL: `CreateBookingRequest`, `PriceQuoteDto`, `IBookingService.PreviewPriceAsync`
3. PL: `BookingsController.PreviewPrice` (AJAX), price box on the form
4. Check (BR-11): 3 hours × $20 = $60; 2 days × 8 × $20 = $320

Cycle 2.6.b — Add booking (FR-11)

1. BLL: `IBookingStateMachine` + `BookingStateMachine` + `Actor` (full transition table, NFR-05)
2. BLL: `CreateBookingValidator` (BR-04 limits, start ≥ now + 24 h, same local day; uses `TimeProvider`)
3. BLL: `BookingMappingProfile`, `CreateAsync` (guide bookable; copy price; local time → UTC by guide `TimeZoneId`; status Pending)
4. PL: `FutureDateAttribute` + adapter; Tourist `BookingsController.Create` GET/POST, `CreateBookingViewModel`, form
5. Check: 1 hour or 11 hours refused; start in 10 hours refused; booking with a non-bookable guide refused; saved booking keeps the price after the guide changes it

2.7 Guide answers requests — FR-13, FR-16

- **BRs:** BR-05, BR-06, BR-08, BR-09 · **NFRs:** NFR-01, NFR-05 · **needs:** FR-11

Cycle 2.7.a — List requests and bookings (FR-16)

1. DAL: `IBookingRepository.GetForGuideAsync`
2. BLL: `BookingDto`, `GetForGuideAsync`
3. PL: Guide `BookingRequestsController.Index` / dashboard (UI screen 3), `GuideDashboardViewModel`
4. Check: guide sees only own requests with status

Cycle 2.7.b — Accept (FR-13)

1. DAL: `IBookingRepository.HasOverlapAsync`
2. BLL: `AcceptAsync` (owner; Pending → Accepted via state machine; no overlap with Accepted/Confirmed; set `PaymentDeadlineUtc`; `RowVersion`)
3. PL: Accept POST button
4. Check (BR-09, BR-08): overlapping accept is refused; deadline = earlier of +24 h and start; another guide cannot accept it

Cycle 2.7.c — Reject (FR-13)

1. BLL: `RejectAsync` (owner; Pending → Rejected)
2. PL: Reject POST button
3. Check (BR-06): only Pending can be rejected; Rejected is final

2.8 Tourist bookings — FR-15

- **BRs:** BR-05, BR-07 · **NFRs:** NFR-01, NFR-05 · **needs:** FR-11

Cycle 2.8.a — List my bookings

1. DAL: `IBookingRepository.GetForTouristAsync`
2. BLL: `GetForTouristAsync`
3. PL: `BookingsController.Index`, `MyBookingsViewModel`, view (UI screen 6) with buttons by status
4. Check: tourist sees only own bookings with correct status

Cycle 2.8.b — Cancel (withdraw)

1. BLL: `WithdrawAsync` (owner; Pending or Accepted → Cancelled, `WithdrawnByTourist`)
2. PL: Withdraw POST button on Pending and Accepted rows
3. Check (BR-07): withdraw works before payment only; Confirmed cannot be withdrawn

2.9 Payment — FR-14

- **BRs:** BR-07, BR-08, BR-12 · **NFRs:** NFR-04, NFR-05 · **needs:** FR-13

Cycle 2.9.a — Checkout details

1. DAL: `IPaymentRepository.GetByBookingAsync`
2. BLL: `CheckoutDto`, `IPaymentService.GetCheckoutAsync` (owner, Accepted, before deadline)
3. PL: Tourist `PaymentsController.Checkout`, `CheckoutViewModel`, view (UI screen 4, marked test mode)
4. Check: checkout opens only for an Accepted booking of this tourist

Cycle 2.9.b — Add payment

1. BLL: `IPaymentGateway` + `SimulatedPaymentGateway`, `CardDetails`, `PaymentValidator` (format only)
2. BLL: `PaymentRequest`, `PaymentReceiptDto`, `PayAsync` (one transaction: `Payment` + Accepted → Confirmed)
3. PL: `Pay` POST, receipt view with `TransactionReference`
4. Check (BR-07, BR-08, NFR-04): pay after the deadline is refused; paying twice is refused; no card data in the database or logs

Phase 3 — Secondary

3.1 Scheduled job — FR-17, FR-18

- **BRs:** BR-06, BR-08, BR-10 · **NFRs:** NFR-05, NFR-06 · **needs:** FR-13, FR-14

Cycle 3.1.a — Auto-cancel (FR-17)

1. DAL: `IBookingRepository.GetDueForAutoCancelAsync`
2. BLL: `IBookingStatusJobService.RunAsync` (Pending > 24 h → `NoGuideResponse`; Accepted past deadline → `NotPaid`), `JobRunSummary`
3. PL: `BookingStatusJob : BackgroundService` (new scope each run, `Interval` from `appsettings.json`)
4. Check: with a short interval and old test rows, both kinds become Cancelled with the right reason

Cycle 3.1.b — Auto-complete (FR-18)

1. DAL: `GetDueForCompletionAsync`
2. BLL: add Confirmed past `EndUtc` → Completed to `RunAsync`
3. Check (BR-10): a Confirmed booking in the past becomes Completed; nothing else changes

3.2 Reviews — FR-19, FR-20

- **BRs:** BR-13, BR-14, BR-15 · **NFRs:** NFR-01 · **needs:** FR-18

Cycle 3.2.a — Add review (FR-19, FR-20)

1. DAL: `IReviewRepository.ExistsForBookingAsync`
2. BLL: `CreateReviewRequest`, `CreateReviewValidator`, `ReviewMappingProfile`, `IReviewService.AddAsync` (+ rating update in one transaction), `CanReviewAsync`
3. PL: Tourist `ReviewsController.Create`, `CreateReviewViewModel`, review pop-up on My Bookings
4. Check (BR-13–15): only own Completed booking; second review refused; average and count update correctly

Cycle 3.2.b — List reviews on the public profile (FR-09)

1. PL: show paged reviews on `GuidesController.Details` (data already from 2.5.a)
2. Check: new review appears on the guide's page with the new average

Phase 4 — Later (only if time is left)

No cycles. These are out of scope in SRS section 6: chat, real payment gateway and refunds, cancelling a Confirmed booking, availability calendar, notifications, other languages, review replies and moderation, admin statistics, tour packages and group tours, suspending a guide, mobile app.

Next: each cycle goes to Build → for example `plan FR-11 Add booking` (cycle 2.6.b).